

SOFTWARE DEVELOPMENT ASSISTANT – ZIMNAT LIFE ASSURANCE**CANDIDATE ASSESSMENT – TECHNICAL ASSESSMENT**

Congratulations on reaching the 1st assessment stage for the Software Development Assistant role. As part of the assessment, you are required to build a Policy Management System (Web & Mobile). From the time you receive this document, kindly work on it on your **own** under the following guidelines:

- Kindly submit your answer; via email to chimbodin@zimnat.co.zw; by **0900hrs on Monday 18 May 2026**
- The format of the presentation, scheduled for a date to be communicated should you be successful, will be as follows:
 - 2–3 minutes video demo showing mobile app functionality
 - 12 minutes to field questions from the Selection Panel
- When you arrive; your presentation will be set up on the projector screen and a laptop ready for you to use

*****IMPORTANT POINTS:**

Please submit:

1. Source code (GitHub or ZIP)
2. SQL file for database setup
3. README file containing:
 - Setup instructions
 - How to run backend and mobile app
 - Assumptions made
 - AI usage disclosure

ASSIGNMENT BACKGROUND

A local life assurance company wants a simple digital system to manage insurance policies and improve communication with clients. Clients currently struggle to:

- View their policy information
- Track renewal dates
- Raise issues with the company

The company now wants:

- A **Web Application** for staff (Admin & Policy Officers)
- A **Flutter Mobile Application** for clients

Objective

Build a simple system that allows:

- Managing insurance policies
- Viewing policy details and renewal dates
- Uploading policy documents
- Clients raising queries from mobile
- Staff responding to queries from web

Technologies

You may use:

Backend (Choose one)

- **Laravel (Preferred)**
- OR

- **Core PHP**

Database

- MySQL / MariaDB

Frontend

- Web: HTML/CSS (basic UI allowed)
- Mobile: Flutter

System Roles

1. Admin (Web Application)

- Manage users
- View all policies and queries

2. Policy Officer (Web Application)

- Manage policies
- Upload documents
- Respond to client queries

3. Client (Mobile Application)

- View policies
- View renewal dates
- View documents
- Raise queries
- View query responses/status

Functional Requirements

1. User Access

- Login for all users
- Role-based access control

2. Policy Management (Web App)

Admin and Policy Officers must be able to:

- Create policies
- View policies
- Edit policies
- Delete policies

Policy Fields

- Policy Number
- Client Name
- Insurance Type
- Premium Amount
- Start Date
- Renewal Date
- Status

Status Values

- Active
- Expired
- Pending Renewal

3. Document Upload

Policy Officers must be able to:

- Upload documents to a policy
- View uploaded documents

Allowed File Types

- JPG
- PNG
- PDF

4. Query System (Mobile + Web)

Client (Mobile App)

- Raise a query
- View query status
- View response from staff

Policy Officer (Web App)

- View all queries
- Respond to queries
- Update query status:
 - Open
 - In Progress
 - Resolved

5. Dashboard

Display simple summaries:

- Total policies
- Active policies
- Expired policies
- Pending queries
- Resolved queries

Technical Requirements

Backend

- PHP 8+
- OOP principles required
- MySQL database
- Proper folder structure
- Input validation
- Basic error handling

Mobile App (Flutter)

- Login system
- View policies
- View policy details
- Raise queries
- View query status and responses

Non-Functional Requirements

We are assessing:

- OOP design
- Code quality and structure
- Database design (relationships must be properly designed using foreign keys)
- Role-based access control
- Security (validation, safe uploads)
- Flutter API integration
- Readable and maintainable code

The demo must show policy view, details, queries being raised and their statuses/responses.